Personal Project — Frontend Application

Project Overview

In this project, you will build a React frontend for the backend personal project you previously completed.

Your goal is to turn your existing FastAPI backend into a usable browser-based application, demonstrate realistic business workflows, and deploy the frontend so it can communicate with your backend.

Business Research & Project Scope

Your frontend must match the business domain and selected business need from your backend project.

If your original website analysis or business/domain research was incomplete, you should revisit the real website and improve your understanding of its users, business needs, and key workflows before designing the frontend.

You do not need to choose a new company or rebuild your backend from scratch.

- Review what the business does, who uses the platform, and which problems your selected feature solves.
- Identify the main user journeys your frontend should support and the data users need to see or enter.
- Ensure the frontend provides a coherent business experience rather than a collection of unrelated API test forms.
- You may update your existing FastAPI endpoints or data models if needed to support the frontend.

Required Technologies

Your frontend project must use:

- React
- React Router
- Redux Toolkit
- Docker
- Nginx

- AWS EC2

Required Frontend Features

Your project must include:

1. **Multiple Pages with React Router**
    - Create meaningful routes and navigation for your business workflow.
2. **Redux Toolkit for State Management**
    - Use Redux Toolkit to manage real application state, not merely install the dependency.
3. **Authentication and/or Authorization Flow**
    - Support login and authenticated application behavior using your backend authentication system.
4. **Protected Routes**
    - Restrict access to pages that require authentication and handle unauthenticated access appropriately.
5. **Backend API Integration**
    - Connect the frontend to your deployed FastAPI backend.
6. **Docker + Nginx + EC2 Deployment**
    - Containerize the frontend, serve it through Nginx, and deploy it to AWS EC2.

Frontend Design Considerations

Plan your frontend around the actual user workflow.

Think about which pages are needed, how users navigate between them, which components can be reused, and how data moves between the browser, Redux state, and the backend.

Design forms and API interactions around your backend contract. The interface should be functional, not just visually complete.

Optional Bonus Features

The following are optional:

- SSE for real-time updates
- WebSocket or Socket.IO for real-time communication
- Improved UI/UX design and responsive mobile-friendly layout
- Search, filtering, sorting, or pagination
- Reusable components and clear component organization
- Strong error handling and user feedback
- Environment-based API configuration

AI Tool Usage

You may use AI coding tools such as Cursor, Claude Code, GitHub Copilot, Codex, or ChatGPT.

However, you are responsible for understanding the code, architecture, and decisions in your project.

Be prepared to explain your React components, routing, Redux state management, authentication, API integration, and deployment.

Presentation Requirements

Date: **Oct 26th at 1 PM ET**
Duration: **20-minute presentation**

Slides are not required. You may demonstrate directly from your deployed website, code, browser developer tools, and backend API documentation.

Your presentation should cover:

1. **Business Context**
    - Briefly introduce the real website, its users, the selected business need, and why the feature matters.
    - If your earlier business/domain research was weak, use this opportunity to present an improved analysis.
2. **Frontend Overview**
    - Walk through the main pages, navigation, and intended user workflow.
3. **Live Demo**
    - Demonstrate login, protected routes, and the main business interactions end to end.
4. **API Integration**
    - Explain how the React application calls your FastAPI backend and handles responses or errors.
5. **State Management**
    - Explain what Redux Toolkit manages and why.
6. **Architecture**
    - Briefly explain key components, routes, and authentication handling.
7. **Deployment**
    - Show that the public frontend is deployed on EC2 using Docker and Nginx, and that it connects to your deployed backend.
8. **Edge Cases**
    - Where possible, demonstrate validation, failed requests, empty states, or other realistic cases.

Final Submission

Submit the **public URL of your deployed frontend website**.

Your deployed application must be accessible for the live demo.

Be prepared to demonstrate the working application on **Monday, October 26, at 1:00 PM ET**.

No separate slide deck is required.

Key Reminder

This project extends your existing backend personal project.

Focus on building a usable frontend that reflects the real business need, integrates correctly with your FastAPI backend, and supports a complete user workflow.

Revisit and strengthen your original business/domain research if needed.